IMAGE PROCESSING

Samuel Falzone

September 2026

Contents

1 Immagini **2**
1.1 Il Concetto di Pixel e la Rappresentazione Spaziale . . . 2
1.1.1 Connettività e Topologia degli Intorni . . . 2
1.1.2 Intorni di Raggio Variabile e Metriche di Distanza . . . 3
1.1.3 Convenzioni di Indicizzazione Matriciale (C vs. Matlab) . . . 4
1.1.4 Dinamica e Rappresentazione dei Livelli di Intensità . . . 4
1.2 Sintesi additiva, Immagini Standard e Struttura Dati . . . 4
1.3 Interpretazione delle immagini . . . 5
1.4 Limiti fisici . . . 5
1.4.1 Sintesi colori Sottrattiva . . . 6
1.5 Filtri . . . 7
1.6 Lo Spazio Colore HSB/HSV e la Percezione Visiva . . . 7
1.7 Dimostrazione Pratica dell'HSB con l'Immagine di Test "Baboon" . . . 8
1.8 Il Cerchio Goniometrico del Colore nello Spazio HSB . . . 8

2 Proprietà delle Immagini di Test Standard e Caratteristiche di Frequenza **9**
2.0.1 Scomposizione dei Canali, Tipi di Dato e Memory Layout . . . 9
2.0.2 Architettura degli Header, Endianness e Ordinamento dei Canali . . . 9

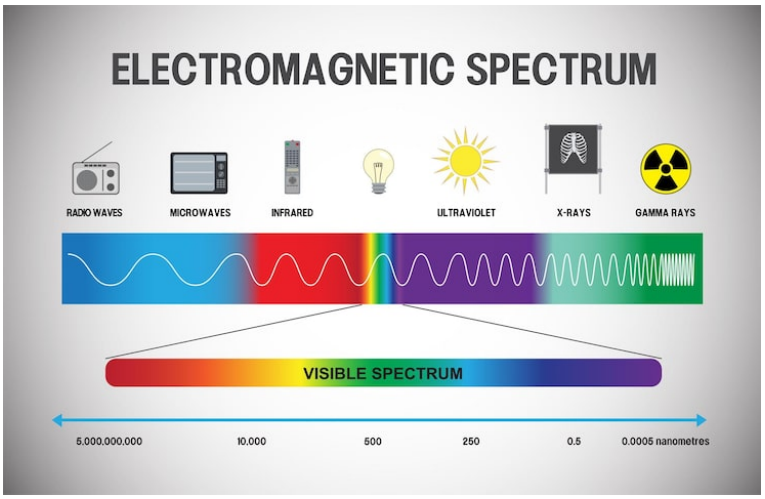
3 Analisi della Registrazione: Tubo Catodico, Trinitron, Spazio YUV **9**
3.1 Sintesi e Trascrizione Dettagliata dell'Audio . . . 9

4 Filtro di convoluzione (Gaussiano) **10**

5 immaggini indicizzate **12**

1 Immagini

Un'immagine può essere definita come una **funzione bidimensionale** $f(x, y)$, dove x e y sono coordinate su un piano e ogni coppia di coordinate x e y rappresenta "l'intensità" o il livello di grigio di un'immagine in un determinato punto; detto ciò definiamo per valori finiti di x, y ed f un'immagine come un'immagine digitale. Ogni immagine digitale è definita da un numero finito di elementi detti **pixel**. Nonostante la vista sia il senso più sviluppato nell'uomo, non si riesce con la sola vista a captare tutto lo spettro delle radiazioni visive dello spettro elettromagnetico.



1.1 Il Concetto di Pixel e la Rappresentazione Spaziale

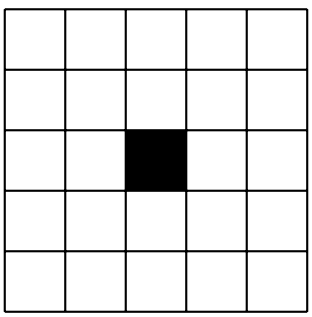
Il termine **Pixel** nasce come contrazione di *Picture Element* (elemento dell'immagine). Dal punto di vista della resa fisica sui display digitali contemporanei, i pixel sono generalmente considerati e costruiti con geometria quadrata. Tuttavia, nell'ambito della codifica e della compressione dei flussi video, i pixel possono essere acquisiti e memorizzati con forme anamorfiche o rettangolari per risparmiare banda:

- L'immagine compressa può essere archiviata con un campionamento asimmetrico (ad esempio stretta e allungata).
- Le corrette proporzioni di visualizzazione (rapporto d'aspetto $4:3$ oppure $16:9$) vengono ripristinate a livello di *rendering* leggendo i metadati memorizzati nell'**header** del file ed eseguendo un ricampionamento mediante appositi **algoritmi di interpolazione** (lineare, bilineare o cubica).

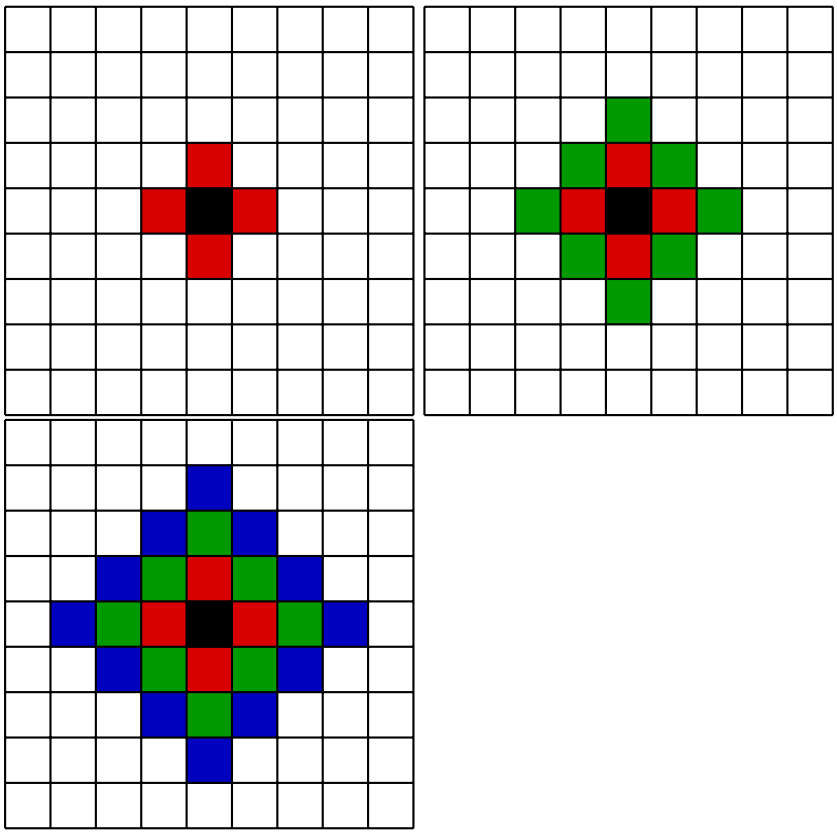
1.1.1 Connettività e Topologia degli Intorni

Dato un pixel centrale p di coordinate (x, y), la definizione di adiacenza spaziale con i pixel circostanti dipende dal modello di connettività adottato:

Un'immagine digitale I è una funzione defunita sullo spazio discreto bidimensionale i cui valori discreti sono detti livelli di grigio g.
I = {(i,jg):i $\in \{0, ..., w-1\}, j \in \{0, ..., h-1\}, g \in \{0, ...G-1\}$}



- **4-connettività (connettività forte)**: considera adiacenti al pixel centrale esclusivamente i 4 pixel che condividono con esso un lato orizzontale o verticale:



$$N_4(p) = \{(x+1, y),\ (x-1, y),\ (x, y+1),\ (x, y-1)\} \tag{1}$$

- **8-connettività (connettività debole)**: include sia i 4 vicini orizzontali/verticali sia i 4 pixel adiacenti lungo i vertici diagonali, per un totale di 8 vicini:

$$N_8(p) = N_4(p) \cup \{(x+1, y+1),\ (x+1, y-1),\ (x-1, y+1),\ (x-1, y-1)\} \tag{2}$$

1.1.2 Intorni di Raggio Variabile e Metriche di Distanza

Estendendo il concetto di vicinato a distanze crescenti dal pixel centrale, si generano geometrie discrete differenti a seconda della metrica [28–31]:

- **Geometria a Diamante (Distanza Manhattan / City-Block)**: In 4-connettività, l'intorno di raggio 1 centrato in p comprende 5 pixel (il centro più i 4 vicini ortogonali), formando una figura a rombo o diamante (equivalente a un quadrato ruotato di 45°) [28–30]. Tale intorno è governato dalla metrica City-Block (D_4 o Taxi-cab), che ammette esclusivamente spostamenti ad angolo retto:

$$D_4(p, q) = |x - s| + |y - t| \tag{3}$$

  dove $q = (s, t)$ individua le coordinate del pixel generico.

- **Geometria Quadrata (Distanza di Chebyshev)**: In 8-connettività, l'intorno di raggio 1 comprende 9 pixel e assume una forma rigorosamente quadrata. La metrica corrispondente (D_8) misura la distanza massima lungo le componenti ortogonali:

$$D_8(p, q) = \max\left(|x - s|,\ |y - t|\right) \tag{4}$$

- **Geometria Circolare e Distanza Euclidea**: Nel piano discreto, un intorno di forma circolare si ottiene applicando la metrica euclidea standard:

$$D_e(p, q) = \sqrt{(x - s)^2 + (y - t)^2} \tag{5}$$

Poiché il calcolo continuo della radice quadrata in virgola mobile comporta un costo computazionale non trascurabile, è possibile ottenere una valida approssimazione discreta di una circonferenza **alternando iterativamente passi a 4-connettività e passi a 8-connettività** ($4 \to 8 \to 4 \to 8 \dots$), evitando del tutto le operazioni trascendenti.

1.1.3 Convenzioni di Indicizzazione Matriciale (C vs. Matlab)

L'indirizzamento degli elementi in memoria differisce in maniera sostanziale tra i vari ambienti di sviluppo:

- **Indicizzazione zero-based (C/C++)**: I vettori e le matrici partono dall'indice 0, per cui una matrice di dimensione $W \times H$ è indicizzata negli intervalli $[0, W-1]$ e $[0, H-1]$.
- **Indicizzazione one-based (Matlab)**: In accordo con il formalismo dell'algebra lineare matriciale, gli indici partono da 1 (l'elemento di posizione $(0, 0)$ non esiste e genera un errore di esecuzione).
- **Inversione Riga-Colonna**: Nelle matrici bidimensionali di Matlab, la convenzione stabilisce che:

$$\text{Matrice}(r, c) \implies r = \text{Indice di Riga (Altezza } H), \quad c = \text{Indice di Colonna (Larghezza } W) \tag{6}$$

  Poiché nel linguaggio comune si indica prima la larghezza e poi l'altezza ($W \times H$), è frequente incorrere in scambi accidentali di indici all'interno dei cicli (*row-column swap*). Per rilevare immediatamente questi bug logici, è buona norma collaudare gli algoritmi su **immagini di test rettangolari** anziché quadrate, poiché su una matrice quadrata ($N \times N$) l'inversione dell'indice non genera eccezioni di fuori limite (*out of bounds*).

1.1.4 Dinamica e Rappresentazione dei Livelli di Intensità

La luminosità acromatica G di ciascun pixel può essere codificata numericamente in due formati principali:

1. **Rappresentazione a Interi (`uint8`)**: Interi senza segno a 8 bit limitati all'intervallo discreto $G \in [0, 255]$, dove 0 identifica il nero puro e 255 il bianco di massima luminosità.
2. **Rappresentazione Normalizzata in Virgola Mobile (`single` / `double`)**: Valori continui normalizzati nell'intervallo compatto [0.0, 1.0], dove 0.0 rappresenta il nero, 1.0 il bianco e 0.5 il grigio medio. Lavorare in virgola mobile durante i passaggi intermedi dell'elaborazione consente di minimizzare gli errori di quantizzazione e la propagazione degli arrotondamenti numerici, posticipando il *cast* a `uint8` solo alla fase finale di memorizzazione o rendering.

‘‘‘

1.2 Sintesi additiva, Immagini Standard e Struttura Dati

Come funziona questa rappresentazione dei colori nello spazio con le tre componenti in **sintesi additiva** con le tre componenti **rosso, verde e blu**? Sostanzialmente è roba vecchia, 1855, Maxwell, quello che ha fatto le equazioni di Maxwell, per intenderci. Utilizzavamo delle immagini standard e l'immagine più standard che c'era era l'immagine di una certa Lena, la conosciamo a memoria (pregi, difetti, contrasto, luminosità) e ci sono o c'erano fino a pochi anni fa dei repository online dove si trovava questa immagine di Lena in tutte le forme (JPEG, PNG, TIF, GIF con 256, 128 o 16 milioni di colori, in scala di grigio). Analizzando l'immagine di Lena ogni pixel ha tre valori interi fra 0 e 255 che rappresentano le **intensità** di rosso, verde e blu (3 byte per pixel), in Matlab lo rappresentiamo con `uint8`. Se separiamo questi 3 byte, otteniamo tre immagini in **scala di grigi**. Un singolo valore indica l'intensità di luminosità per quel canale. I colori invece necessitano allo stato attuale di una terna di valori tra 0 e 255. Nell'ordine, il primo è destinato al **rosso**, il secondo al **verde** e il terzo al **blu**. Storicamente ci sono stati problemi sull'ordine dei byte (Intel vs Motorola), e persino sulle codifiche delle immagini (RGB per Intel/Windows e BGR per Apple/Mac), per cui nell'intestazione dei file (header) si specifica l'ordine dei canali per evitare errori di miscelazione dei colori.

Matematicamente, un'immagine a colori TrueColor viene rappresentata definendo in ogni posizione spaziale (x, y) un **vettore colore tridimensionale** $\mathbf{c}(x, y)$:

$$\mathbf{c}(x, y) = \begin{bmatrix} R(x, y) \\ G(x, y) \\ B(x, y) \end{bmatrix} \tag{7}$$

dove le tre funzioni $R(x, y)$, $G(x, y)$ e $B(x, y)$ rappresentano le immagini monocromatiche di intensità nei rispettivi canali.

Nello **spazio colore RGB**, il dominio dei colori ammissibili costituisce un cubo unitario cartesiano $[0, 1]^3$, oppure discreto $\{0, \ldots, 255\}^3$ nel caso di quantizzazione a 8 bit per canale:

- L'origine $(0, 0, 0)$ corrisponde al **Nero**.
- Il vertice opposto $(255, 255, 255)$ corrisponde al **Bianco**.
- La diagonale principale che congiunge $(0, 0, 0)$ a $(255, 255, 255)$ contiene tutti i punti di pari intensità $R = G = B$, rappresentando l'asse della **scala dei grigi**.
- La combinazione additiva a coppie dei colori primari genera i colori secondari:

$$\text{Ciano (C)} = \text{Verde} + \text{Blu} = (0, 255, 255) \tag{8}$$
$$\text{Magenta (M)} = \text{Rosso} + \text{Blu} = (255, 0, 255) \tag{9}$$
$$\text{Giallo (Y)} = \text{Rosso} + \text{Verde} = (255, 255, 0) \tag{10}$$

Abbiamo quindi la possibilità di rappresentare la bellezza di 256*256*256 colori diversi che equivalgono a 2^{24} per la terna RGB, 16 milioni di colori rappresentabili digitalmente, e viene definita immagine TRUE-COLOR, anche se venivano chiamate così le immagini a 65 mila colori.

Noi utenti finali utilizziamo 8 bit per canale, 3 canali vuol dire **24 bit per pixel**, ma le imprese cinematografiche o professionali in generale, lavorano a **48 bit per pixel**, quindi 2 byte per canale, con un'elevatissima risoluzione il quale permette di fare il down sampling garantendo un ampio margine di errore e perdita di informazioni, e possono passare da 16 a 8 bit per canale. In realtà si può lavorare anche con 4 byte per pixel, e quindi 32 bit, considerando il 4° canale come l'**alfa-channel**, ovvero il canale che determina la *trasparenza* del pixel.

1.3 Interpretazione delle immagini

Come si potrebbe risolvere il problema del **riconoscimento delle immagini**, per esempio nel caso dell'IA nel riconoscimento dei volti, o nel caso dell'inseguimento dei missili col bersaglio ecc...? Il seguente problema trova risoluzioni con diversi approcci:

- **Approccio antropomorfo**, ovvero l'approccio che basa il suo funzionamento sul simulare, imitare il comportamento umano, ragionamento, visione, analisi ecc...
- **Approccio pragmatico**, più ingegneristico per intendersi, tenta una risoluzione diretta.

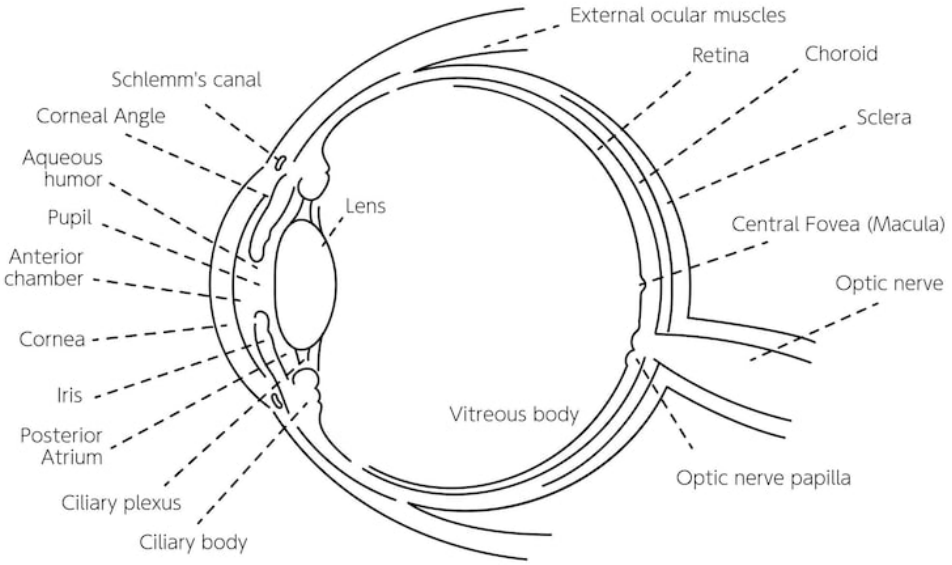
1.4 Limiti fisici

Il nostro occhio è un sensore con un'elevatissima capacità di dettaglio. Il **CCD** è un **sensore luminoso**, sostanzialmente è una matrice di **elementi fotosensibili** (pixel); più sensori sono presenti più è dettagliata l'immagine. L'occhio è chiaramente superiore a un CCD perché ci permette di acquisire le immagini in tante condizioni di luminosità (attualmente). Poi non percepiamo l'immagine in pixel ma come se fosse *continua* (anche se il continuo non esiste, ma si lavora per quanti). Attualmente i cellulari possono essere dotati di diverse camere per consentire diverse fasce di luminosità per CCD. La luminosità si misura in **LUMEN** ovvero $\frac{\text{CANDELE}}{m^2}$. In assenza di luce o in diminuzione di quest'ultima iniziamo a perdere la visione dei colori, prima del blu, poi del rosso e poi del verde e iniziamo a vedere in **visione scotopica**, ovvero in **scala di grigi**.

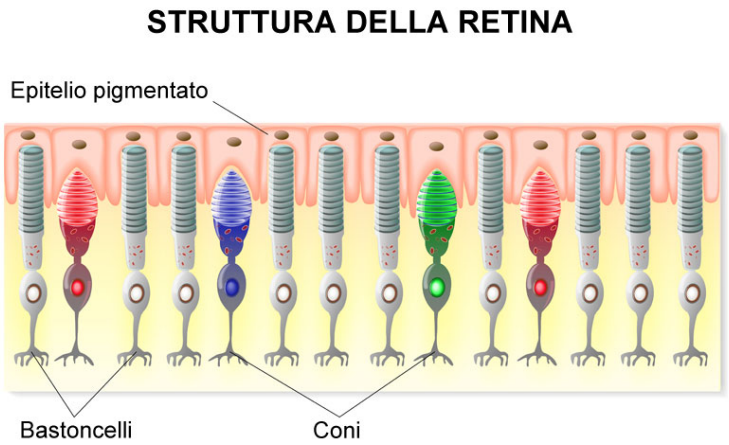
Sole	10^9 cd/m^2
Faro abbagliante di automobile	10^7 cd/m^2
Strada a mezzogiorno	10^5 cd/m^2
Cielo diurno	10^4 - 10^6 cd/m^2
Lampada fluorescente	10^3 - 10^4 cd/m^2
Luna piena	10^3 - 10^4 cd/m^2
Visione fotopica (percezione colori)	10 cd/m^2
Visione scotopica (percezione luminosità)	0.01 cd/m^2
Illuminazione stradale	1 cd/m^2
Cielo notturno con luna piena	0.01 cd/m^2
Cielo notturno senza luna	10^{-6} - 10^{-3} cd/m^2

È vietata qualsiasi diffusione, anche parziale e gratuita, del presente materiale.
Questo supporto alla didattica non sostituisce i testi consigliati dal docente. A.A. 2025-2026

Com'è fatto il nostro occhio?



È costituito da una lente che attraverso i muscoli ottici mette "a fuoco" in un punto chiamato **fovea**; aprendosi e chiudendosi permette l'attraversamento di un fascio luminoso. Se questo fascio luminoso non raggiunge la fovea si è in presenza di problemi di vista che si risolvono con l'occhiale, che sostanzialmente devia l'angolazione del fascio luminoso per colpire appunto la fovea, che è costituita chiaramente da **fotorecettori**. Più in basso troviamo i nervi, dove non sono presenti i fotorecettori o ce ne stanno meno. Chiaramente questa variazione è bilanciata da entrambi gli occhi per i punti ciechi e dal cervello, che combina i due emisferi per comporre un'immagine "continua". I vertebrati si dicono di conseguenza a **visione foveata**, ovvero siamo dotati di fotorecettori solo nel punto di focus. Spostandosi di qualche grado, la percezione diminuisce del 50%; in realtà noi vediamo una zona non sfocata anziché un punto perché la nostra pupilla compie dei micro spasmi, *movimento saccadico*. Il nostro occhio in media può percepire un massimo di 50 sfumature di grigio, considerando che abbiamo a disposizione 256 sfumature. Stessa cosa per la terna RGB, essendo che percepiamo meno distinzioni rispetto al grigio. La struttura della nostra retina è composta da diversi elementi come **coni e bastoncelli**. I bastoncelli ci permettono di percepire le diverse sfumature di grigio e sono numericamente superiori ai coni, mentre i coni si dividono i valori RGB. Chiaramente i colori che percepiamo peggio sono le sfumature di blu, poi di rosso e poi di verde.



1.4.1 Sintesi colori Sottrattiva

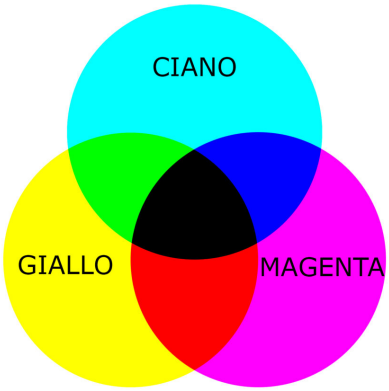
Sostanzialmente quella che ci hanno insegnato a scuola, ovvero che la mescolanza di tutti i colori produce il **nero**, o comunque un grigio molto scuro, lo stesso spettro colori che si mette nelle stampanti: Ciano,

Magenta, Giallo, Nero. La combinazione di questi colori primari produce quelli dello spettro colore in sintesi SOTTRATTIVA:

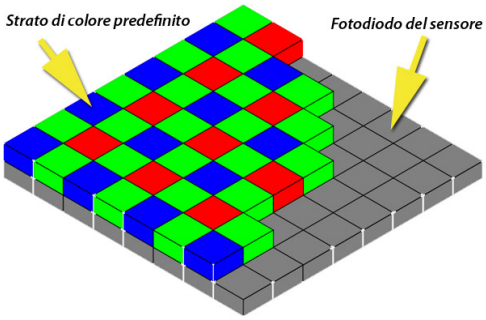
- Magenta + Giallo = Rosso
- Giallo + Ciano = Verde
- Ciano + Magenta = Blu
- Ciano + Magenta + Giallo = Grigio Scuro prossimo al nero

1.5 Filtri

Un CCD abbiamo detto essere un **sensore luminoso**; considerando un'immagine 7x7 costituita quindi da CCD, abbiamo un'immagine in scala di grigi, in quanto il fotorecettore può definire solo la percentuale di luminanza e non di colore. Se volessimo rappresentare il colore con CCD di diverso colore dovremmo rappresentare l'immagine con dimensione 21x7. Per non far ciò il trucco consiste nell'applicare un filtro sul fotorecettore, ovvero una pellicola sul CCD. Questo filtro si chiama filtro di Bayer; ogni CCD percepisce la sfumatura di un certo valore tra Rosso, Verde, Blu.



Di conseguenza ogni sensore può percepire unicamente la percentuale di un solo colore; questo viene risolto desumendo la percentuale dalla media dei colori adiacenti per poi successivamente rappresentare l'immagine. Ovviamente esistono diversi filtri; lo stesso filtro può essere applicato in 8 posizioni distinte girandolo e rivoltandolo. La media di desunzione dei valori adiacenti agisce come un **filtro passa basso**, che permette di far passare solo le basse frequenze. Un filtro passa basso non fa passare i dettagli, il risultato sarà la **sfocatura dell'immagine**. Ci sono anche i **filtri passa banda** e i **filtri passa alto** (descritti successivamente). Il filtro di Bayer, che è utilizzato nella quasi totalità dei sistemi di acquisizione, ci permette di riconoscere l'eventuale modifica di immagini, perché i colori non rispettano più il valore della media di desunzione dei pixel vicini.



Il rover marziano, ovvero il robottino che è stato mandato sulla crosta di Marte per condurre degli studi, ha un filtro di Bayer speciale applicato al suo sistema di cattura immagini: la maggior parte della "pellicola" è verde, poi una parte minore rossa e ancora meno blu!

1.6 Lo Spazio Colore HSB/HSV e la Percezione Visiva

Oltre al modello RGB, viene introdotto lo spazio colore **HSB** (*Hue, Saturation, Brightness*), noto anche nelle sue varianti geometriche a cono singolo o doppio cono sovrapposto (HSI/HSV) [1]. Tale modello offre importanti vantaggi percettivi e computazionali:

- **Descrizione intuitiva del colore**: Quando si deve descrivere un colore nel linguaggio naturale, non si specificano le percentuali numeriche della terna RGB, ma si indica la tonalità precisando quanto essa sia più o meno "carica" (*Saturation*) e quanto sia luminosa (*Brightness*) [1].
- **Conservazione dell'informazione**: Lo spazio HSB è uno spazio tridimensionale; pertanto, la conversione da RGB ad HSB non comporta alcuna perdita irreversibile di informazione [2].

- **Disaccoppiamento tra colore e luminosità**: Nello spazio RGB le tre componenti cromatiche fondamentali sono mescolate all'informazione di luminosità [2]. In RGB, i valori di grigio puro si collocano esclusivamente lungo la diagonale che congiunge i due vertici opposti del cubo del colore (dal nero al bianco), dove le componenti assumono valori identici su tutti e tre i canali (ad esempio la sfumatura di grigio $(33, 33, 33)$) [2].
- **Corrispondenza con la fisiologia umana**: L'occhio umano possiede recettori retinici distinti: tre tipologie di **coni** dedicati rispettivamente al rosso, al verde e al blu, affiancati dai **bastoncelli** responsabili della percezione della luminosità in scala di grigi [3]. Rispetto all'RGB, lo spazio HSB modella meglio questa separazione biologica disaccoppiando il canale della luminanza dalle componenti cromatiche [3].
- **Formulazione analitica**: Le relazioni matematiche per convertire le coordinate da HSB ad RGB si fondano su trasformazioni trigonometriche elementari basate su funzioni seno e coseno [3, 4].

1.7 Dimostrazione Pratica dell'HSB con l'Immagine di Test "Baboon"

Per illustrare visivamente l'effetto dei singoli parametri dello spazio HSB, viene impiegata l'immagine campione storica del babbuino (*Baboon*), caratterizzata da una tessitura complessa (la peluria) e da regioni con tinte molto sature e contrasti accentuati tra rosso, celeste e verde [5].

- **Tonalità (*Hue*) e Cerchio Goniometrico**: La tonalità identifica la componente pura del colore mediante un angolo definito nell'intervallo continuo da $0°$ a $360°$ (ovvero da 0 a 2π radianti) [5]. La variazione della tonalità corrisponde geometricamente alla rotazione del cilindro dei colori [6]:
  - A $0°$ non viene applicata alcuna variazione rispetto all'immagine d'ingresso [7].
  - Nel cerchio goniometrico i colori primari sono collocati a intervalli di $120°$: a $0°$ si trova il rosso, a $120°$ il verde e a $240°$ il blu [7].
  - Applicando una rotazione di $+120°$, tutti i pixel rossi dell'immagine assumono una colorazione verde [7].
  - Applicando una rotazione di $+240°$, i pixel originariamente rossi diventano blu [7].
  - Modificando l'angolo in senso orario o antiorario, l'intera palette trasla: le sfumature rosse si spostano verso il giallo, il giallo assume la tinta del verde e così via [6].
- **Saturazione (*Saturation*)**: Definisce quanto il colore sia carico o puro [1, 8]. Azzerando completamente la saturazione, l'informazione cromatica scompare del tutto, restituendo un'immagine pura in scala di grigi [8]. Impostandola al valore massimo, le tinte diventano estremamente accese e cariche di colore [8].
- **Luminosità (*Brightness*)**: Regola l'intensità luminosa del pixel, variando lungo l'asse che va dal buio completo (nero) fino alla massima brillantezza (bianco) [8].

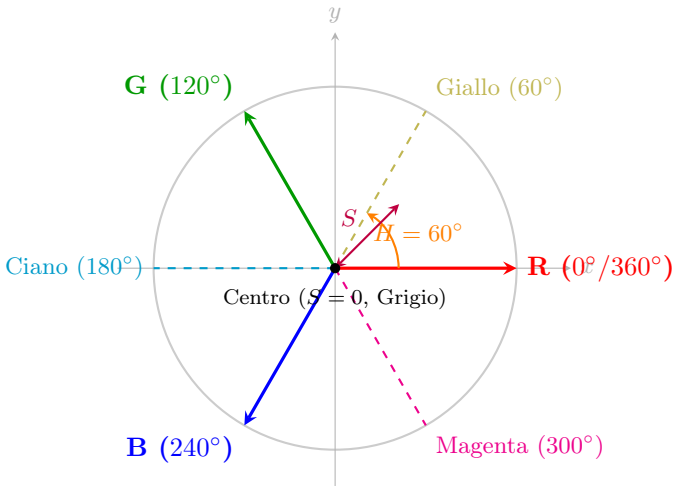
1.8 Il Cerchio Goniometrico del Colore nello Spazio HSB

Sezionando il solido tridimensionale dello spazio HSB ad un livello prefissato di luminosità (B) e saturazione massima ($S = 1$), si ottiene una circonferenza unitaria coincidente con un **cerchio goniometrico**.

I tre colori primari della sintesi additiva (R, G, B) risultano equispaziati lungo il perimetro con una separazione angolare di $120°$ ($\frac{2}{3}\pi$ radianti):

- **Rosso (R)**: asse di riferimento orizzontale a $0° \equiv 360°$.
- **Verde (G)**: ruotato di $120°$ in senso antiorario.
- **Blu (B)**: ruotato di $240°$ in senso antiorario.

I colori secondari si collocano esattamente nelle posizioni intermedie: il Giallo a $60°$, il Ciano a $180°$ e il Magenta a $300°$. La Tonalità (*Hue*) individua l'angolo polare H, mentre la Saturazione (S) rappresenta la distanza radiale dal centro acromatico ($S = 0$).



Rappresentazione polare sul cerchio goniometrico del modello HSB: la Tonalità (H) è la coordinata angolare, la Saturazione (S) è la coordinata radiale, e i tre primari RGB sono disposti a intervalli di 120°.

2 Proprietà delle Immagini di Test Standard e Caratteristiche di Frequenza

Dal punto di vista dell'analisi ed elaborazione delle immagini, l'ampia diffusione di *Lena* e delle altre immagini citate deriva dalle loro marcate proprietà spettrali e topologiche:

1. **Dettagli ad Alta Frequenza Spaziale**: Il cappello con piume di *Lena* e la trama della pelle del babbuino contengono frequenze spaziali elevate.
2. **Aree di Gradiente Liscio**: Le spalle e il volto offrono transizioni di contrasto morbide.
3. **Pattern Periodici e Texture (*Barbara*)**: L'immagine include motivi a strisce per lo studio dei fenomeni di *aliasing*.
4. **Superfici Lucide e Saturazione (*Peppers*)**: Riflessi speculari e confini nitidi tra componenti cromatiche.

2.0.1 Scomposizione dei Canali, Tipi di Dato e Memory Layout

In memoria, un'immagine TrueColor a 24 bit viene allocata come una matrice tridimensionale di dimensioni $M \times N \times 3$ con tipo di dato `uint8`:

$$f(x, y) \in \{0, 1, 2, \ldots, 255\} \quad \text{per ciascun canale } R, G, B \tag{11}$$

2.0.2 Architettura degli Header, Endianness e Ordinamento dei Canali

Le immagini memorizzate su disco comprendono due parti fondamentali:

1. **Header (Intestazione del File)**: Struttura dati binaria che specifica dimensioni, profondità di bit, compressione e ordine dei canali.
2. **Data Payload**: La sequenza dei valori dei pixel.

3 Analisi della Registrazione: Tubo Catodico, Trinitron, Spazio YUV

3.1 Sintesi e Trascrizione Dettagliata dell'Audio

- **Fisica del Tubo a Raggi Catodici (CRT) e Scansione Raster**:

  - Un display CRT è costituito da un tubo sottovuoto dove un cannone elettronico spara elettroni sui **fosfori**.
  - La scansione avviene in modalità **raster** (riga per riga).

- **Persistenza dei Fosfori e Retinica**: Fonde i singoli impulsi luminosi in un'immagine fluida.
- **Standard di Alimentazione**:
  - **Stati Uniti (NTSC)**: Frequenze di Rete e Frame Rate: Negli Stati Uniti la rete elettrica a 60 Hz ha imposto uno standard televisivo a 30 fps (frame al secondo), mentre in Europa la rete a 50 Hz ha portato lo standard a 25 fps (nel cinema si usano invece 24 fps, storicamente 23.976 fps)
  - **Europa (PAL)**: Vantaggio Europeo di Risoluzione: Il passaggio da 30 a 25 fps ha consentito in Europa un risparmio di banda di circa 1/627. Questo margine è stato sfruttato per aumentare la risoluzione verticale dello schermo: dalle 240 righe dello standard americano (320×240) si è passati a 256 righe (320×256), offrendo una qualità visiva migliore
  - **Il Televideo (Teletext)**: I primi schermi avevano ampolle rotonde (derivate dai radar militari), alle quali veniva applicata una cornice rettangolare $4:3$ per mascherare i bordi2728. Le righe esterne nascoste (nel Vertical Blanking Interval) sono state riutilizzate per trasmettere segnali digitali, dando origine al servizio di Televideo
- **Scansione Interlacciata vs. Progressiva**: L'interlacciamento trasmette prima le righe dispari e poi le pari per risparmiare banda.
- **Spazio Colore YUV**:

$$YUV \implies Y = \text{Luminanza}, \quad U = \text{Differenza Blu}, \quad V = \text{Differenza Rosso} \tag{12}$$

4 Filtro di convoluzione (Gaussiano)

Un filtro di convoluzione, per esempio un filtro passa basso, sfoca l'immagine, mentre un filtro passa alto enfatizza i dettagli e i contrasti. Il filtro di Gauss è una funzione a variabili separabili $k_{ij} = k'_i \otimes k'_j$.

```
lena = imread("/matlab drive/lena.png");
>> img = rgb2gray(lena);
>> figure; imshow(img);
```



$\Rightarrow$



— da recuperare 40/50 minuti

formato dycom

blocco testo Il secondo parametro di imshow è un vettore che ci permette di specificare il range della scala di grigio, se non specifichiamo il range in automatico prende i valori minimi e massmi dell'mmagine, altrimenti si può specificare il range e può essere diverso in quel cao si deve fare la proporzione .

$$u' = \left[255 \cdot \frac{(\text{rangeMassimo} - \text{rangeMinimo}) - \text{rangeMinimo}}{\text{rangeMassimo} - \text{rangeMinimo}} \right] \tag{13}$$

Come si fa il ribilanciamento pixel per pixel?, con un lookup tableo. Operazione puntuale = loockup table

Listing 1: Esempio di script MATLAB

```
>> game = imread("/MATLAB Drive/lena.png");
>> img = rgb2gray(lena);
>> figure; imshow(img, [0 255])
```

Listing 2: Esempio di script MATLAB

```
>> tmp = single(img);
>> tmp = uint8(255*(tmp-min(tmp(:)))/(max(tmp(:)) . min(tmp(:))));
>> tmp = uint8(255*(tmp-min(tmp(:)))/(max(tmp(:)) - min(tmp(:))));
```

loock up table

Listing 3: Esempio di script MATLAB

```
    function angolo(img)

        [bordo, ~, dy, dx] = edge(rgb2gray(img), 'sobel');
        angolo = atan2(-dy, -dx);
        direzione = round(359*(angolo+pi)/(2*pi))+1;
        direzione(bordo==0) = 0;

        figure; imshow(img, []);
        figure; imshow(direzione, [[0 0 0] ; hsv(360)]);
```

5 immaggini indicizzate

un'immaggine si definisce indicizzata se per ogni... , si definiscono a partire da tavolozze. Le immaggini a scala di grigio matlab le vede molto bene senza problemi blablabla un'immaggine in scala di grigio la possiamo immagginare come un imaggine indicizzata a scala di grigio appunto, dove 0 corrisponde a [0 0 0] 1 corrisponede a [1 1 1] e così via la tavolozza in matlabese si chiama cmap

Listing 4: Esempio di script MATLAB

```
>> tavolozza = (0:1/254:1)'; tavolozza = [tavolozza tavolozza tavolozza];
```

—- lezione del 15, poca roba, algoritmo Ekbert, median catcolor quantisation prende il parallelepipedo , prende un lato con la misura più lunga, lo taglia a "meta". ottenendo due parallelepipedi adiacenti, tagliando a meta, se non trova il colore, applica dinuovo lo scringt, e itera finquanto no si ottiene il numero di parallelepipedi che coincide col numero di colori di cui abbiamo bisogno, raggiunto l'obbiettivo, per determinare la scelta in funzione delle varie classi di parallelepipiedo scelte si considera il mediano.

Per indicizzare un immaggine su matlab

```
>> figure;imshow(lena);
>> figure;imshow(lena2,lenalut);
>> figure;imshow(lena2,baboonlut);
>> figure; imshow(babbon2, lenalut);
```

» —-